HARP: Hardware-Based Pseudo-Tiling for Sparse Matrix Multiplication Accelerator

Jinkwon Kim
KAIST
Daejeon, Republic of Korea
coco@kaist.ac.kr

Myeongjae Jang
KAIST
Daejeon, Republic of Korea
myeongjae0409@kaist.ac.kr

Haejin Nam
KAIST
Daejeon, Republic of Korea
haejinnam@kaist.ac.kr

Soontae Kim
KAIST
Daejeon, Republic of Korea
kims@kaist.ac.kr

ABSTRACT

General sparse matrix-matrix multiplication (SpGEMM) is a memory-bound workload, due to the compression format used. To minimize data movements for input matrices, outer product accelerators have been proposed. Since these accelerators access input matrices only once and then generate numerous partial products, managing the generated partial products is the key optimization factor. To reduce the number of partial products handled, the state-of-the-art accelerator uses software to tile an input matrix. However, the software-based tiling has three limitations. First, a user manually executes the tiling software and manages the tiles. Second, generating a compression format for each tile incurs memory-intensive operations. Third, an accelerator that uses the compression format cannot skip ineffectual accesses for input matrices.

To overcome these limitations, this paper proposes hardware-based pseudo-tiling (HARP), which enables logical tiling of the original compressed matrix without generating a compression format for each tile. To this end, HARP utilizes our proposed Runtime Operand Descriptor to point to an effectual column-row pair in a pseudo-tile. Consequently, HARP enables a user to use the accelerator as a normal SpGEMM operation does without tiling. Furthermore, HARP does not require a compression format for each tile and can skip ineffectual accesses for input matrices. To further improve the efficiency of pseudo-tiling, HARP performs super-tiling to combine pseudo-tiles and sub-tiling to further tile a pseudo-tile. Experiment results show that HARP achieves 4×, 35×, and 33× speedup and 5×, 664×, and 233× energy efficiency on average compared to the state-of-the-art outer product accelerator, CPU (MKL), and GPU (cuSPARSE), respectively.

CCS CONCEPTS

• **Hardware** → **Emerging architectures**; **Hardware accelerators**; • **Computer systems organization** → **Architectures**.

KEYWORDS

Sparse Matrix, Sparse matrix multiplication, SpGEMM, Spare matrix tiling, Tiling, Hardware accelerator, Application-specific hardware

MICRO '23, October 28–November 01, 2023, Toronto, ON, Canada

ACM ISBN 979-8-4007-0329-4/23/10…$15.00
https://doi.org/10.1145/3613424.3623790

ACM Reference Format:
Jinkwon Kim, Myeongjae Jang, Haejin Nam, and Soontae Kim. 2023. HARP: Hardware-Based Pseudo-Tiling for Sparse Matrix Multiplication Accelerator. In *56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '23), October 28–November 01, 2023, Toronto, ON, Canada.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3613424.3623790

1 INTRODUCTION

General sparse matrix-matrix multiplication (SpGEMM) is a fundamental operation used in various application domains such as graph algorithms [8, 20, 66, 74], machine learnings [21, 36, 47, 61], and high-performance computing [6, 9, 13, 69]. Since sparse matrices contain numerous zero elements (e.g., Amazon co-purchasing network consists of a $400K \times 400K$ matrix with $3.2M$ non-zero elements, density$\approx 2 \times 10^{-5}$[43]), they are commonly stored in a compression format to eliminate memory accesses to zero elements. However, due to the additional compression metadata required by such a format, SpGEMM operations have low arithmetic intensity (i.e., operations divided by memory accesses (flops/bytes)), and thus become a memory-bound workload in the current system.

To minimize data movements for input matrices in SpGEMM operations, outer product-based accelerators have been proposed [22, 53, 57, 70, 75], as shown in Figure 1 (A). Although Figure 1 illustrates dense matrix formats for simplicity, it is worth noting that sparse matrices are stored in a compressed format, requiring decoding for element access. As shown in Figure 1 (A), outer product accelerators sequentially access column-row pairs (①), generate partial product(s) (i.e., the output(s) of element-wise multiplication(s)), and merge them with previous partial products (②) to generate the output matrix (③). Since the partial products are repeatedly accessed for merging, the on-chip buffer is used to store them. However, since the number of partial products is usually larger than the on-chip buffer size, the on-chip buffer management is the key optimization factor.

To store all partial products in the on-chip buffer, the state-of-the-art outer product accelerator [22] relies on software preprocessing to tile the input matrix A, as shown in Figure 1 (B) (①). However, the software-based tiling has three limitations. First, a user needs to manually run the tiling software to tile the input matrix A and then provide the tiling information (e.g., the number of tiles and the address of each tile) to the accelerator. Second, the software-based tiling calculates the precise tiling boundaries in matrix A and then generates a compression format for each tile, resulting in many memory-intensive operations. Third, a compression format does not provide any data skipping information for the other input matrix, which leads to ineffectual accesses unless additional skipping hardware is employed. For example, in Figure 1 (B) (②), since the third and fourth columns for tile 0 in A are zero columns, accessing

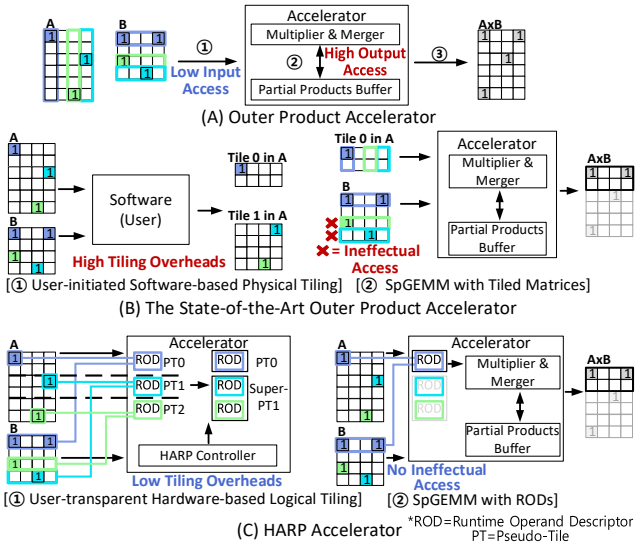

Figure 1: Overviews of (A) outer product, (B) the state-of-the-art outer product [22], and (C) HARP accelerators.

the third and fourth rows of matrix B is ineffectual and thus can be skipped. However, a compression format cannot provide this skipping information.

To overcome the aforementioned three limitations of the software-based tiling, we propose user-transparent Hardware-based Pseudo-tiling (HARP). HARP has two key design concepts implemented in hardware. First, as shown in Figure 1 (C) (①), instead of generating a compression format for each tile using software, HARP logically tiles the input matrix A as if it were tiled, while preserving the original compression format for the input matrix. Consequently, a user can execute the accelerator without considering tiling. In this paper, we refer to this tiling as pseudo-tiling. To realize pseudo-tiling, we propose Runtime Operand Descriptor (ROD) to point to an effectual column-row pair in a particular pseudo-tile. By utilizing RODs, HARP not only accesses effectual column-row pairs in a pseudo-tile but also can naturally skip the ineffectual accesses. For instance, in Figure 1 (C) (②), HARP accesses only the first column-row pair by using a ROD when multiplying pseudo-tile 0, eliminating the two ineffectual accesses.

Second, instead of calculating precise tiling boundaries, HARP calculates the boundaries of the initial pseudo-tiles by dividing the number of the rows in the input matrix by the fixed number of pseudo-tiles in hardware (e.g., PT0~PT2 in Figure 1 (C) (①)). Subsequently, to improve the accuracy of the boundaries of pseudo-tiles, HARP additionally adjusts the boundaries of the initial pseudo-tiles by performing super-tiling and sub-tiling (e.g., super-PT1 in Figure 1 (C) (①)). Super-tiling expands the boundary of a pseudo-tile by combining adjacent pseudo-tiles while sub-tiling further partitions the boundary of a pseudo-tile into smaller sub-pseudo-tiles. Consequently, the accuracy of pseudo-tiling in HARP with super-tiling and sub-tiling becomes close to that of the precise software-based tiling (the average relative error rate is 5.8%).

Our experiments show that HARP achieves 4×, 35×, and 33× speedup and 5×, 664×, and 233× energy efficiency on average compared to the state-of-the-art outer product accelerator [22], CPU (MKL), and GPU (cuSPARSE), respectively, on the University of Florida sparse matrices [12].

Our key contributions are summarized as follows:

- We qualitatively and quantitatively analyze the overheads of the software-based tiling.
- We propose hardware-based pseudo-tiling (HARP). To access elements in a pseudo-tile without generating the compression formats for tiles, HARP introduces a runtime operand descriptor (ROD).
- To improve the accuracy of pseudo-tiling, we propose super-tiling to expand the boundary of a pseudo-tile and sub-tiling to further partition the boundary of a pseudo-tile.

This paper is organized as follows: Section 2 provides background and motivations. Section 3 explains the proposed HARP. Section 4 presents experimental results. Section 5 further discusses HARP. Section 6 summarizes related works. Section 7 concludes the paper.

2 BACKGROUND AND MOTIVATION

2.1 Sparse Matrix Compression Formats

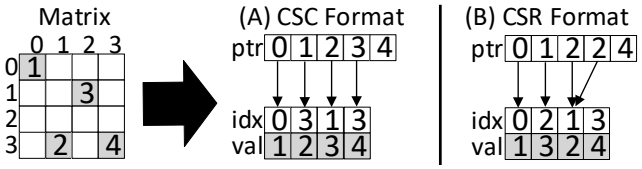

Figure 2: (A) CSC, and (B) CSR format examples.

Several compression formats have been proposed to eliminate zero elements in sparse matrices [7, 29, 65]. Figure 2 shows the most widely used formats: Compressed Sparse Column (CSC) and Compressed Sparse Row (CSR). The CSC format consists of three arrays: pointer (*ptr*), row index (*idx*), and non-zero value (*val*). The *idx* and *val* arrays store the row indices and non-zero values in the column-major order while the *ptr* array stores the starting offsets for *idx/val* pairs in each column. For instance, in Figure 2 (A), the first column in the matrix has one non-zero element; therefore, one *idx/val* pair is stored, and the *ptr* array stores *(0)* and *(1)*. Thus, the sizes of *ptr* and *idx/val* arrays are proportional to the number of columns and non-zero elements in the matrix, respectively. Unlike the CSC format, which uses the column-major order, the CSR format encodes the matrix in the row-major order.

2.2 Outer Product Algorithm and Accelerator

General sparse matrix-matrix multiplication (SpGEMM) consists of 'sparse matrix'-'dense vector/matrix' multiplication, 'sparse matrix'-'sparse matrix' multiplication, etc. In this paper, we focus on the 'sparse matrix'-'sparse matrix' multiplication, as it is the most challenging one because both input matrices are compressed. Note that our motivation and proposed HARP can be applied to all SpGEMM operations.

The outer product algorithm has been proposed to compute SpGEMM operations. As shown in Figure 3 (A), the outer product algorithm consists of multiply and merge phases. In the multiply phase, each column-row pair (i.e., a column in matrix A and the corresponding row in matrix B) is multiplied to generate partial products. Throughout this paper, to visualize column-row pairs, we represent the values of a column-row pair using the same color. For example, since the first column-row pair has two multiplications in outer product operation, two partial products are generated. Subsequently, in the merge phase, the generated partial products are accumulated element-wise to form the output matrix. Therefore, during the outer product algorithm, the input matrices A and B are accessed only once, whereas many partial products are accessed to generate the output matrix.

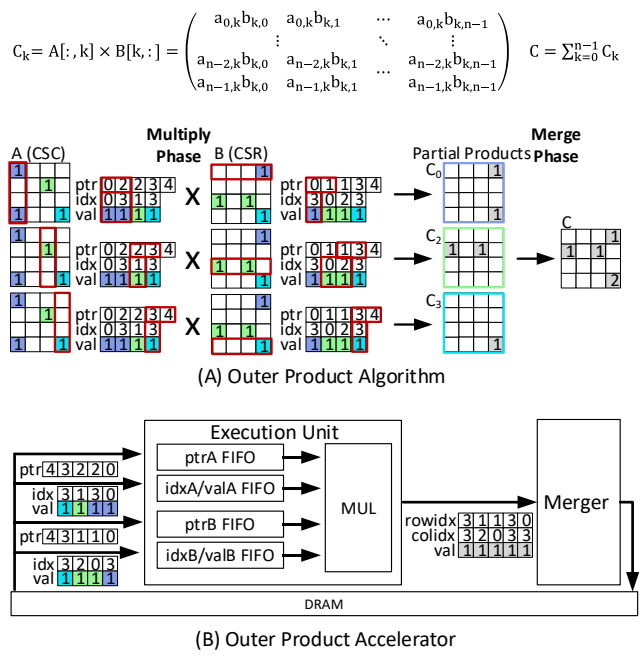

Figure 3: (A) Outer product algorithm and (B) an outer product accelerator.

Outer product accelerators [22, 53, 57, 70, 75] have been proposed to implement the outer product algorithm in hardware. Outer product accelerators consist of execution units with four FIFO queues and a multiplier and mergers to accumulate the partial products, as shown in the example accelerator with one execution unit and merger in Figure 3 (B). Because input matrices A and B are sequentially accessed in the column- and row-major orders, they are stored in CSC and CSR formats for sequential decompression, respectively. Based on this property, the input fetching hardware for outer product accelerators is implemented by four FIFO queues (*ptrA*, *idxA/valA*, *ptrB*, and *idxB/valB*), and can be launched with the addresses and sizes of each array. Thus, the interface of outer product accelerators can be easily implemented by the memory-mapped I/O interface without any ISA support [22]. Followed by the FIFO queues, the multiplier generates one partial product with row and column indices in each cycle. Subsequently, the merger sorts the partial products and accumulates them.

2.3 Limitations of the Software-Based Tiling

Outer product accelerators require many additional DRAM accesses if the partial products buffer cannot store all partial products. Therefore, for on-chip merging without additional heavy DRAM traffic, the state-of-the-art outer product accelerator (Spaghetti [22]) tiles the input matrix A by software. As shown in Figure 4, the software-based tiling consists of three procedures: CSC2CSR, horizontal tiling, and vertical tiling procedures. Note that since horizontal and vertical tiling procedures rely on both matrices A and B, these procedures need to be executed whenever either of the input matrices is modified.

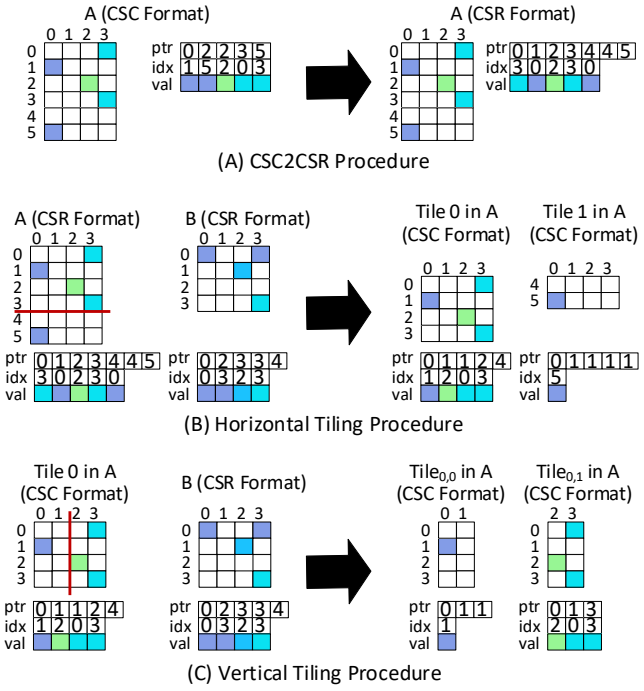

Figure 4: An example of the software-based tiling [22].

CSC2CSR Procedure: To achieve precise tiling, the software-based tiling sequentially accesses each row in matrix A. However, because outer product accelerators use the CSC format for matrix A, matrix A should be converted into the CSR format. The CSC2CSR procedure sequentially reads the CSC format and then copies the *idx/val* element into the proper location in the CSR format [14, 68]. Consequently, many random memory accesses occur during this procedure.

Horizontal Tiling Procedure: To find the precise horizontal tiling boundaries in matrix A, the horizontal tiling procedure accesses each row in matrix A sequentially and several corresponding rows in matrix B to calculate the number of partial products generated in each row. Based on the horizontal tiling boundaries, a compression format for each tile is generated by copying the proper elements. Therefore, memory-intensive random accesses and copy

operations are required. In Figure 4 (B), we assume that the partial products buffer size is four. Thus, two tiles are generated because one produces four partial products and the other produces two.

Vertical Tiling Procedure: The vertical tiling procedure distributes an equal number of effectual execution operations to the execution units for load balancing [17, 19, 22, 48, 52, 64]. In this example, Tile 0 in matrix A is partitioned into two tiles with the same number of effectual multiplications. In this paper, we assume that the vertical tiling procedure does not create separate compression formats for each tile. Instead, this procedure calculates the offsets of *ptrA*, *idxA*, and *valA* arrays for each tile (e.g., $\text{Tile}_{0,0}$ and $\text{Tile}_{0,1}$) in the horizontal tile (e.g., Tile 0).

Unfortunately, the software-based tiling has three limitations. First, as explained in Section 2.2, outer product accelerators use the addresses and sizes of input matrices as inputs. As a result, a user needs to manually run the software-based tiling and then manage and schedule each tile in matrix A to achieve optimal performance. (e.g., memory allocations for each tile and cache flushing in the host CPU to invalidate the data for tiles after tiling [62].)

Second, the software-based tiling incurs many memory-intensive operations for precise tiling and generating a compression format for each tile. Figure 5 shows the overheads of the software-based tiling in the Spaghetti accelerator [22]. Note that the overhead for the software-based tiling is the sum of CSC2CSR, horizontal tiling, and vertical tiling. To estimate quantitative overheads for the software-based tiling, we generate CPU traces of it using Intel Pin tool [49] and then carry out Ramulator [35] CPU trace mode. For system configuration, we model Intel i9-9900K with 5GHz and HBM2 with 2Gbps. The average execution time overhead is 56.9% of the total execution time. (Further discussed in Section 4.3)

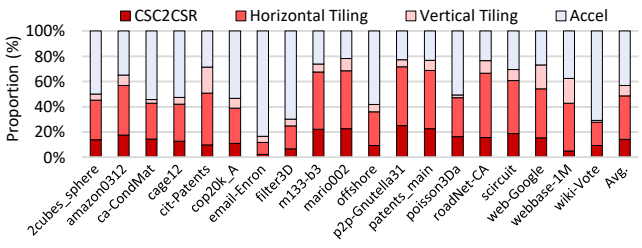

Figure 5: The execution time breakdown of Spaghetti accelerator [22].

Third, because outer product accelerators utilize the conventional CSC and CSR format for input matrices, they cannot skip ineffectual accesses. For example, as shown in Figure 6 (A), since the second column of $\text{Tile}_{0,0}$ in A and the third row in matrix B are zero, accessing the second row in matrix B and the first column in $\text{Tile}_{0,1}$ is unnecessary for computation. Figure 6 (B) shows the breakdown of DRAM read traffic during accelerator execution in Spaghetti [22] (i.e., a more detailed breakdown of *Accel* part in Figure 5). IneffectualA and IneffectualB denote the number of ineffectual accesses in matrices A and B, respectively, due to the presence of a zero row in matrix B and a zero column in matrix A. If the non-zero elements of a column in matrix A are concentrated in some tiles, tiling can exaggerate the number of ineffectual accesses in matrix B. As a result, IneffectualB can constitute the majority of the DRAM traffic

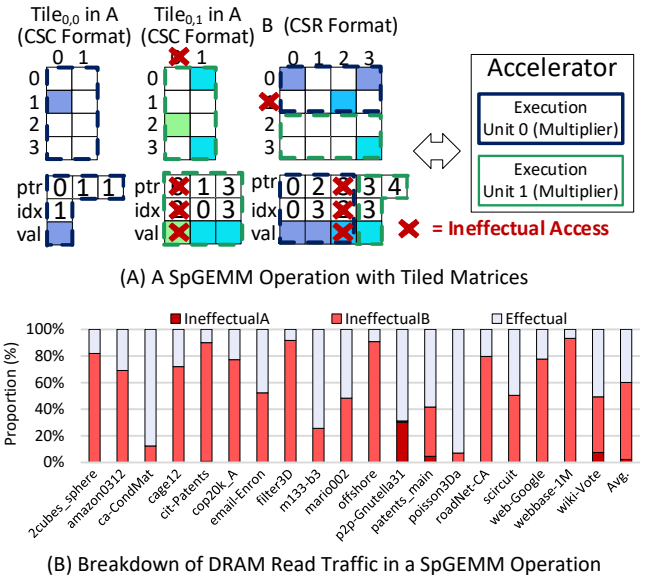

Figure 6: (A) An example and (B) breakdown of DRAM read traffic in a SpGEMM operation with tiled matrices.

during accelerator execution. On average, 60.1 % of read traffic is wasted in a SpGEMM operation with tiled matrices[1].

In summary, the software-based tiling is not user-friendly and incurs huge overheads and many ineffectual accesses.

3 HARDWARE-BASED PSEUDO-TILING

3.1 Overview

HARP has two key design concepts implemented in hardware. First, HARP tiles the input matrix logically while preserving its original compression format, rather than generating a compression format for each tile as the software-based tiling does. Second, HARP roughly calculates the horizontal boundaries of initial pseudo-tiles and then adjusts them by super-tiling and sub-tiling. Based on the horizontal boundaries of pseudo-tiles, HARP utilizes Runtime Operand Descriptor (ROD) to access an effectual column-row pair in a particular pseudo-tile. A ROD points to an *idxA/valA* element and all *idxB/valB* elements in a column-row pair and is dynamically updated to point to the next *idxA/valA* element. Consequently, HARP has one ROD for each column-row pair and can transfer it across pseudo-tiles. In this paper, we use the notation *ptrA/idxA/valA* and *ptrB/idxB/valB* to represent elements in the *ptr/idx/val* arrays in matrices A and B, respectively. By comparing the boundaries of pseudo-tiles and an *idxA* element pointed to by a ROD, HARP can properly access the elements of a particular tile. To implement the pseudo-tiling on hardware in a user-transparent manner, HARP consists of *Pseudo-Tiling Stage*, *Super-Tiling Stage*, *SpGEMM Stage*, and an optional *Sub-Tiling Stage*.

[1]In Section 5.1, we discuss the impact of skipping hardware on Spaghetti. The skipping hardware inspects *ptrA* and *ptrB* elements to eliminate ineffectual accesses. However, the speedup for Spaghetti with skipping hardware is 1.2× on average due to the indirect accesses and high overheads of the software-based tiling as shown in Figure 5.

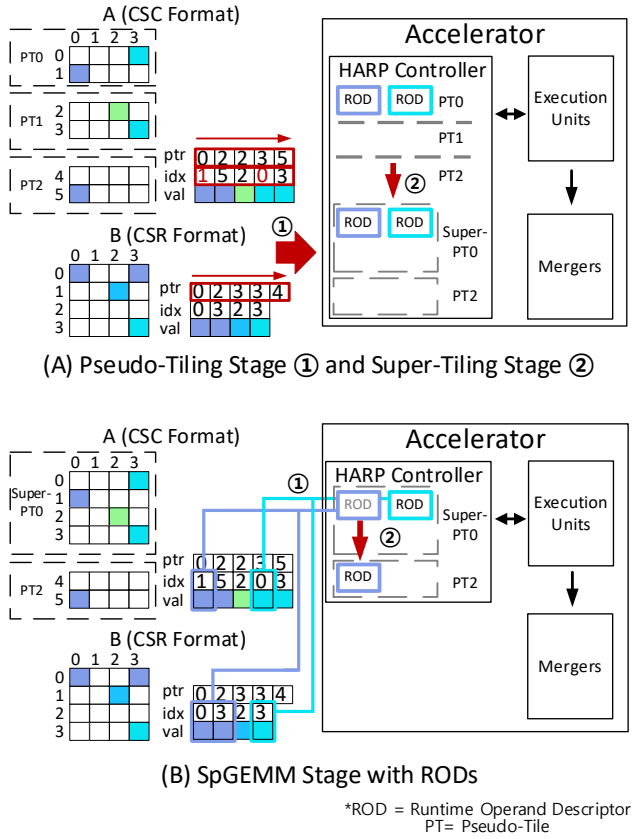

Figure 7: Overview of HARP.

Figure 7 (A) shows *Pseudo-Tiling Stage* and *Super-Tiling Stage* in HARP using the same matrices as in Figure 4. In *Pseudo-Tiling Stage*, HARP sequentially accesses and analyzes the *ptrA*, *idxA* and *ptrB* arrays only once to generate RODs. By sequentially analyzing the *ptrA* and *ptrB* arrays, HARP can easily identify zero columns and zero rows in matrices A and B, respectively, and does not generate RODs for these pairs (i.e., what i^{th} and $i+1^{th}$ elements in the *ptrA* array are identical means a zero column in matrix A.). For instance, in Figure 7 (A), the RODs for the second and third column-row pairs are not generated.

After a ROD is generated, based on the first *idxA* element pointed to by the ROD and the boundaries of the initial pseudo-tile, HARP determines the starting pseudo-tile for the ROD. To calculate the horizontal boundaries of the initial pseudo-tiles, HARP divides the number of the rows in matrix A by the fixed number of pseudo-tiles in hardware. For instance, in Figure 7 (A), the six rows of matrix A are evenly divided into three predefined pseudo-tiles. As a result, the blue and cyan RODs belong to the pseudo-tile 0 because their first *idxA* elements are *(1)* and *(0)*, respectively. After all RODs are initialized, HARP conducts the *Super-Tiling Stage* to combine the boundaries of adjacent pseudo-tiles if the total number of partial products generated in these pseudo-tiles does not exceed the partial products buffer size, as shown in Figure 7 (A) (②).

Subsequently, HARP performs *SpGEMM Stage* with RODs as shown in Figure 7 (B). In this stage, HARP utilizes RODs in each pseudo-tile, starting from the lowest index to the highest index. HARP compares the *idxA* element pointed to by the ROD with the boundaries of pseudo-tiles. If pseudo-tiles are combined into a super-pseudo-tile, the boundaries of the constituent pseudo-tiles are combined into a single boundary. Subsequently, HARP dynamically updates the ROD to point to the next *idxA/valA* element. If the next *idxA* element belongs to another pseudo-tile, HARP transfers the ROD to that pseudo-tile. For instance in Figure 7 (B) (①), the blue ROD is transferred to pseudo-tile 2 after completing operations with that ROD for super-pseudo-tile 0. In contrast, the cyan ROD is not transfered to pseudo-tile 1 because pseudo-tiles 0 and 1 are combined. Additionally, HARP optionally conducts *Sub-Tiling Stage* if further tiling of a pseudo-tile is required.

In summary, HARP provides user-transparent hardware-based tiling, eliminates overheads for generating the compression format in each tile and can skip ineffectual accesses.

3.2 Detailed Descriptions of HARP

To realize pseudo-tiling, HARP has a *Pseudo-Tiling Context* comprised of multiple entries along with a *CombineFlag*, as shown in Figure 8. Each entry stores three fields for a pseudo-tile: the number of RODs in the pseudo-tile (*RODNum*), the number of partial products to be generated in the pseudo-tile (*PP.Num*), and a ROD FIFO to store a few RODs. When the ROD FIFO is full, RODs are sequentially stored in the reserved DRAM area. To dynamically point to an *idxA/valA* element and to access all *idxB/valB* elements in a column-row pair, a ROD consists of four pointers: (1) the current pointer and (2) the end pointer to *idxA/valA* elements of the corresponding column-row pair, and (3) the begin pointer and (4) the end pointer to *idxB/valB* elements of the corresponding column-row pair. A ROD can be initialized by sequentially analyzing the elements of *ptrA*, *idxA*, and *ptrB* arrays. HARP sequentially performs *Pseudo-Tiling Stage*, *Super-Tiling Stage*, *SpGEMM Stage*, and an optional *Sub-Tiling Stage*.

(1) Pseudo-Tiling Stage: In this stage, HARP calculates the horizontal boundaries of the initial pseudo-tiles by dividing the number of the rows in matrix A by the fixed number of pseudo-tiles in hardware. Based on these boundaries, each entry of the *Pseudo-Tiling Context* is updated by sequentially analyzing the elements of *ptrA*, *idxA*, and *ptrB* arrays.

The detailed process of *Pseudo-Tiling Stage* is shown in Figure 8. In cycle 0, HARP analyzes the first *idxA* element of the first column-row pair (① and the green boxes). Because the *idxA* element *(1)* is within the boundary of the pseudo-tile 0, HARP updates the corresponding entry in the *Pseudo-Tiling Context* (②). Consequently, a ROD is generated by copying the *ptrA/ptrB* elements of the first column-row pair; *RODNum* and *PP.Num* are increased by one and two, respectively, because a new ROD is inserted and the column-row pair in the pseudo-tile will generate two partial products. In cycle 1, the second *idxA* element of the first column-row pair is analyzed (③), and then *PP.Num* is incremented by two without the ROD insertion in the pseudo-tile 2 (④). Note that the blue ROD in pseudo-tile 0 will be transferred to pseudo-tile 2 during *SpGEMM Stage*. In cycles 2 and 3, because the second column in matrix A

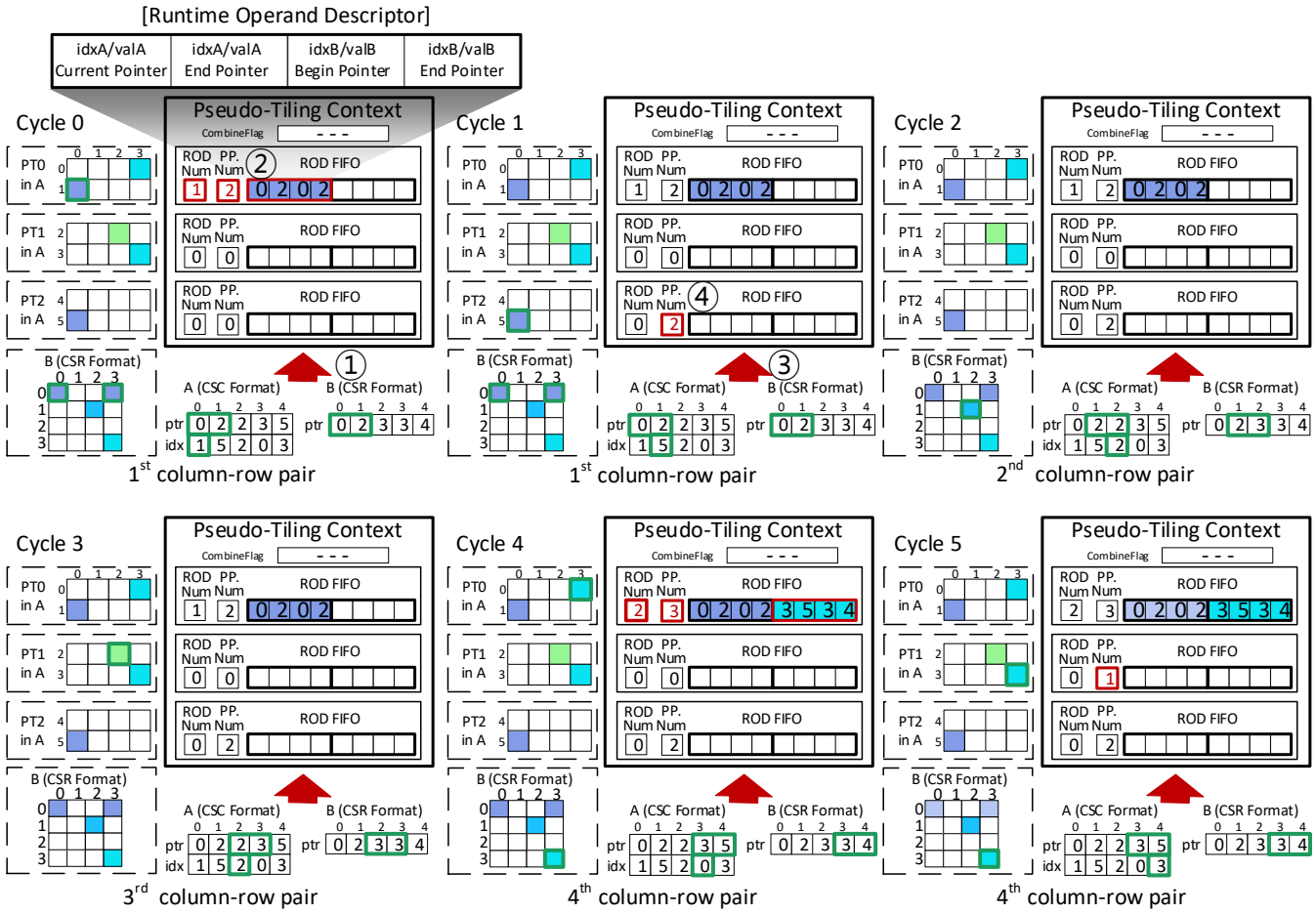

Figure 8: Pseudo-Tiling Stage in HARP.

and the third row in matrix B have no element, HARP skips the operation for the second and third column-row pairs, respectively. Note that HARP naturally skips ineffectual accesses for matrices A and B by not generating RODs in those cases. In cycles 4 and 5, the process for the fourth column-row pair is similar to that for the first column-row pair. Note that since *Pseudo-Tiling Stage* handles one non-zero element in matrix A per cycle, the total cycles are approximately proportional to the size of *idxA* array.

(2) Super-Tiling Stage: Outer product accelerators need to redundantly read matrix B for each tile in matrix A. Thus, the number of tiles in matrix A determines the number of accesses to matrix B. Since the boundaries of the initial pseudo-tiles are roughly calculated, certain pseudo-tiles should be combined to reduce the number of pseudo-tiles. To achieve this, HARP sequentially analyzes *PP.Num* in each pseudo-tile from the lowest index and then combines some pseudo-tiles if possible. To manage super-tiling, each pseudo-tile has one bit of *CombineFlag* and stores the flipped bit of the previous pseudo-tile's *CombineFlag* if they are not combined.

In Figure 9, we assume that the partial products buffer size is four. Thus, pseudo-tiles 0 and 1 are combined into super-pseudo-tile 0 because the sum of the number of their partial products does not exceed four. Thus, the *CombineFlag* bits for pseudo-tiles 0 and 1 are *(0 0)* because they are combined, whereas the *CombineFlag* bit for pseudo-tile 2 is *(1)*. because it is not combined with pseudo-tile 1. Based on the *CombineFlag*, HARP manages the boundaries of super-pseudo-tiles. Note that there is no ROD transfer between pseudo-tiles belonging to the super-pseudo-tile and a ROD is transferred to another pseudo-tile if the *idxA* exceeds the boundary of a super-pseudo-tile.

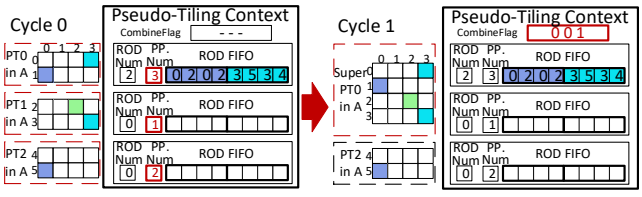

Figure 9: Super-Tiling Stage in HARP.

(3) SpGEMM Stage: In this stage, HARP conducts the conventional outer product operations with RODs. Starting from pseudo-tile 0, HARP sequentially uses one ROD from the pseudo-tile and then reduces the *RODNum* associated with that pseudo-tile by one. If the current pseudo-tile has been combined, the RODs of the pseudo-tile with the lowest index in the super-pseudo-tile are sequentially handled first. After the operations indicated by a ROD are done, the current pointer to *idxA/valA* in the ROD is increased by one to indicate the next element as long as it is smaller than the end pointer to *idxA/valA*. If the next element belongs to the other pseudo-tile, the ROD is transferred accordingly. *SpGEMM Stage* is processed until the *RODNum* of the last pseudo-tile reaches zero.

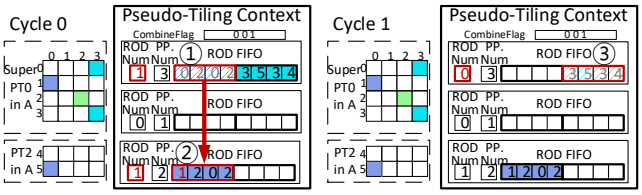

Figure 10: SpGEMM Stage in HARP.

For instance, as shown in Figure 10, the ROD *(0 2 0 2)* in the first pseudo-tile of the super-pseudo-tile 0 is handled in cycle 0 (①). After the operations indicated by the ROD are done, the ROD with the increased current pointer to *idxA/valA* is inserted into the pseudo-tile 2 (②) because the current pointer of the ROD indicates the non-zero element in the pseudo-tile 2. In cycle 1, the next ROD *(3 5 3 4)* is handled. All non-zero elements in this column-row pairs pointed to by the ROD are included in the super-pseudo-tile 0. Thus, there is no ROD transfer and the *RODNum* is decreased by one after the operations are done (③). Subsequently, the ROD *(1 2 0 2)* in the pseudo-tile 2 are handled.

(4) Sub-Tiling Stage: The number of partial products in a particular pseudo-tile can exceed the on-chip buffer size if non-zero elements are concentrated (e.g., the power-law matrix [10]). To solve this problem, HARP has an optional *Sub-Tiling Stage* that further partitions a pseudo-tile as needed. *Sub-Tiling Stage* is triggered whenever HARP encounters a pseudo-tile with the *PP.Num* that exceeds the on-chip buffer size during *SpGEMM Stage*. The purpose of *Sub-Tiling Stage* is to generate additional *Pseudo-Tiling Context* for sub-tiles. Therefore, in this stage, HARP performs a process similar to that of *Pseudo-Tiling Stage* and then conducts *Super-Tiling Stage* and *SpGEMM Stage* for sub-tiles. After *SpGEMM Stage* with sub-tiles are finished, HARP returns to the original *Pseudo-Tiling Context* and then continues *SpGEMM Stage* with the next pseudo-tile. If another sub-tiling is required during *SpGEMM Stage* with the original *Pseudo-Tiling Context*, *Sub-Tiling Stage* is re-activated. Therefore, HARP can support multiple sub-tiling. In addition, HARP also can support recursive sub-tiling by storing one of the pseudo-tiling contexts in the DRAM and then restoring it.

Figure 11 illustrates the example of the *Sub-Tiling Stage*. We assume that the partial products buffer size is two. Thus, because the pseudo-tile 0 exceeds the buffer size, it is partitioned into two sub-pseudo-tiles. Before sub-tiling, the RODs of the overflowed pseudo-tile are flushed into the reserved DRAM area (①). Subsequently, the

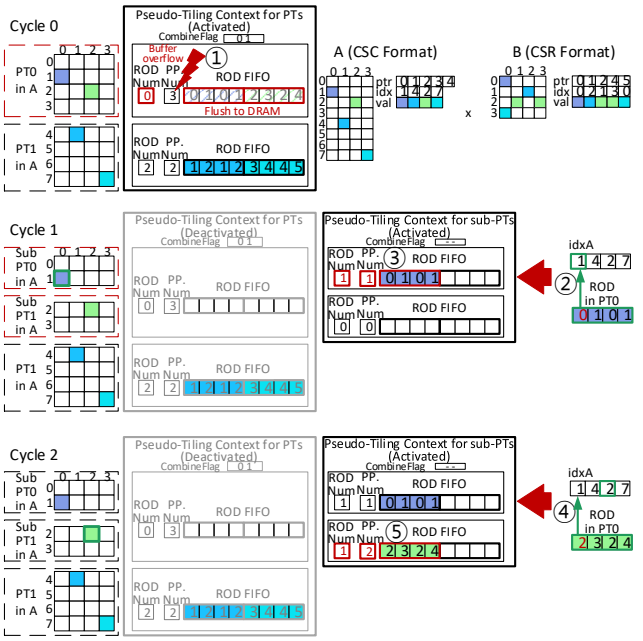

Figure 11: Sub-Tiling Stage in HARP.

Pseudo-Tiling Context for sub-tiles is generated by leveraging RODs from the overflowed pseudo-tile and *idxA* elements pointed to by RODs, instead of *ptrA*, *idxA*, and *ptrB* array used in *Pseudo-Tiling Stage*. In this example, in cycle 1, after the ROD *(0 1 0 1)* is received, the memory address of *idxA* elements for the ROD is calculated from the current pointer to *idxA/valA* in the ROD (②). Based on the ROD *(0 1 0 1)* and the *idxA* element *(1)*, the *Pseudo-Tiling Context* in sub-pseudo-tile 0 is updated[2]. Consequently, the ROD is stored in the sub-pseudo-tile 0, and *RODNum* and *PP.Num* are increased by one (③). In cycle 2, based on the ROD *(2 3 2 4)* and the *idxA (2)* (④), the ROD is stored and *RODNum* and *PP.Num* are updated (⑤).

3.3 Overall Architecture

Figure 12 illustrates the block diagram and example of HARP architecture. HARP can be seamlessly incorporated into an outer product accelerator by adding a HARP controller and slightly modifying execution units. For simplicity, in this example, *SpGEMM Stage* without sub-tiling and one *Pseudo-Tiling Context* for pseudo-tiling are shown, and we assume that DRAM access latency is 1 cycle and internal signals are immediately transferred. HARP consists of a HARP controller, execution units with two FIFO queues and a multiplier, and conventional outer product mergers. Unlike previous accelerators as explained in Figure 3 (B), the execution unit in HARP only includes *idxA/valA* and *idxB/valB* FIFO queues, because RODs point to the accessed elements in a pseudo-tile instead of

[2]In contrast to *Pseudo-Tiling Stage*, an *idxA* element can be outside the boundaries of sub-pseudo-tiles. In this case, the ROD with the modified current pointer to *idxA/valA* is inserted into the pseudo-tile pointed to by the ROD. As a result, the remaining operations indicated by the ROD can be handled after *Sub-Tiling Stage* is completed. There is no ROD transfer between sub-pseudo-tiles and pseudo-tiles during *SpGEMM Stage* using sub-pseudo-tiles.

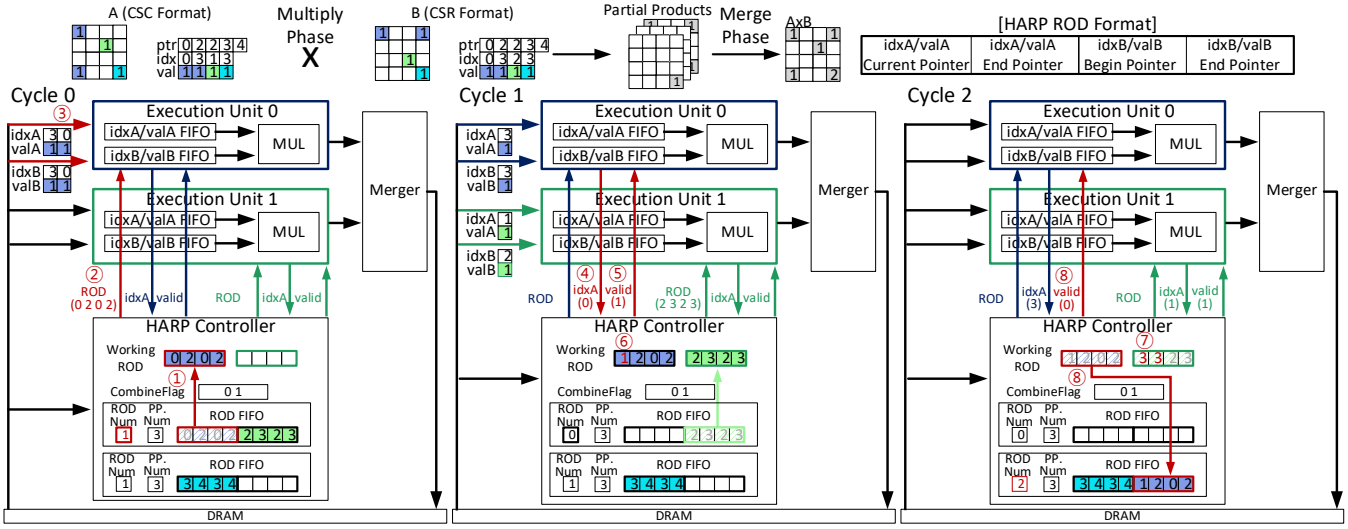

Figure 12: The block diagram and operation example of HARP architecture.

ptr arrays. The HARP controller contains two *Pseudo-Tiling Contexts* for pseudo-tiling and sub-tiling and a working ROD list. The working ROD list temporarily stores the RODs being used in the execution unit. The HARP controller schedules idle execution units in a fine-grained manner by allocating RODs. As a result, HARP can dynamically adjust the load balancing of execution units at runtime, effectively achieving the effect of vertical tiling. Concurrently, the ROD FIFO in the current pseudo-tile prefetches RODs from the reserved DRAM area.

As shown in Figure 12 cycle 0, the HARP controller allocates the ROD *(0 2 0 2)* into the execution unit 0 by transferring the ROD to the first entry of the working ROD list (①) and execution unit 0 (②). Based on the ROD, execution unit 0 reads the *idxA/valA* and *idxB/valB* (③). In cycle 1, execution unit 0 receives the *idxA* element and then sends that element to the HARP controller (④). Subsequently, the HARP controller compares the value of that element with the pseudo-tile boundaries. There are two cases depending on the value of an *idxA* element. First, if an *idxA* element is within the current pseudo-tile boundary, the valid signal is set to *(1)* to allow the execution unit to perform multiplications (⑤), and the current pointer to *idxA/valA* in the ROD is increased by one to indicate the next element (⑥). Subsequently, the increased current and the end pointers to *idxA/valA* in the ROD are compared and the ROD is deleted when they are the same (⑦ in cycle 2) (i.e., the element of the column in matrix A reaches the end). Second, if an *idxA* element is out of the current pseudo-tile boundary, the valid signal is set to *(0)*, and the ROD is moved to the next pseudo-tile calculated from the value of the index element (⑧ in cycle 2). Note that the current pointer to *idxA/valA* in the ROD is not changed, as this element must be re-accessed in the next pseudo-tile.

4 EVALUATION

4.1 Methodology

To evaluate HARP, we incorporate HARP into the state-of-the-art outer product accelerator (Spaghetti [22]) by adding a HARP controller and slightly modifying execution units. To analyze the performance of Spaghetti and HARP, we implement an in-house cycle-accurate simulator with Ramulator. Each hardware component (e.g., execution units and mergers) in accelerators is based on the Spaghetti open-source Chisel code [3]. Therefore, each hardware component has *clock* method to update internal signals and is connected by a hand-shaking protocol (i.e., *decoupled* in chisel) to simulate stall cycles. To evaluate 'sparse matrix'-'sparse matrix' multiplication, we use the 19 most widely used sparse matrices from the University of Florida [12]. The densities of these matrices are within $10^{-3} \sim 10^{-6}$.

Table 1: System configurations

Accelerator Specification	
Execution Unit	*ptrA*, *idxA*, *valA*, *ptrB* FIFO, 256 entries *idxB*, *valB* FIFO, 5000 entries Single-precision floating-point multiplier with 1Ghz
Merger	Sort-merger with 128K depth
Output Buffer	*rowIdx*, *colIdx*, *val* FIFO, 256 entries
Main Memory	HBM2 4GB 2Gbps 8 channel, 2 pseudo channel 4 bankgroup, 4 bank tCL-tRCD-tRP-tRAS: 14-14-15-33 FR-FCFS scheduler ChRoBaRaCo address mapping
CPU Specification	
Intel i7-6850K 6 Cores/12 T 3.6Ghz 4 channel DDR4-2133Mhz oneMKL 2022, -O3, openmp5 multiThreading	
GPU Specification	
NVIDIA A100 cuda 11.7 cuSPARSE library	

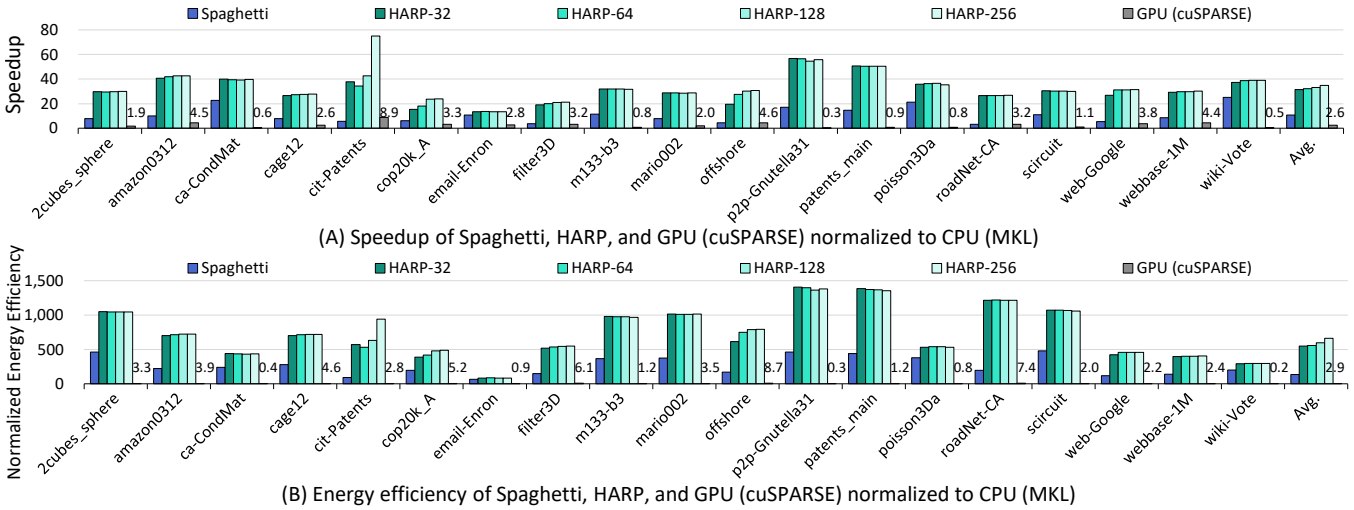

Figure 13: (A) Speedup and (B) energy efficiency of Spaghetti, HARP, and GPU (cuSPARSE) normalized to CPU (MKL).

Table 1 summarizes the accelerator configuration. We adopt the optimal configuration used in [22] (i.e., 32 execution units and 16 mergers) to model the Spaghetti and HARP accelerators. To ensure the robustness of HARP, we sweep the number of initial pseudo-tiles from 32 to 256. In this paper, we use the term HARP-X, where X represents the number of initial pseudo-tiles. In addition to the hardware modules in Table 1, HARP requires a HARP controller. The hardware components of the HARP controller are as follows: First, HARP requires SRAM components to store two *Pseudo-Tiling Contexts* for pseudo-tiling and sub-tiling. Thus, *RODNum* (4bytes), *PP.Num* (4bytes × merger num), and a ROD FIFO (16bytes × 2) for each pseudo-tile and sub-pseudo-tile are needed. Second, HARP has a working ROD list (16bytes × execution unit num = 512bytes). Third, since *Pseudo-Tiling Stage* and *Sub-Tiling Stage* use FIFOs to temporarily store *ptrA*, *idxA*, and *ptrB* for generating a *Pseudo-Tiling Context*, HARP uses 32-, 64-, and 32-entry FIFOs, respectively (total size is 512bytes). Fourth, a HARP controller logic is required to control each stage in HARP. As a result, HARP-32 and HARP-256 need 7KB and 52KB SRAM along with a HARP controller logic, respectively. Note that because HARP does not require *ptrA* and *ptrB* FIFO queues in execution units, the total SRAM size (52KB) for HARP-256 is smaller than the SRAM size for *ptrA* and *ptrB* FIFOs (64KB) in Spaghetti.

To measure the power consumption of accelerators, we use CACTI-P with 45nm for SRAM components [45] (e.g., ROD FIFOs in a HARP controller, FIFO in an execution unit, partial products buffer in a merger, etc.) and the 45nm synthesis results in [18] for single-precision floating-point multipliers and adders. To evaluate the control logic in a HARP controller, we implement it in Chisel [3] and then synthesized with Synopsys Design Compiler with the TSMC 45nm technology [30, 38]. For DRAM power consumption estimation (ACT-PRE/RD/WR/REF/BG), we use the DRAM power equations in DRAMsim3 [46] with HBM2 configuration.

To compare HARP with multiple platforms, we experiment on CPU and GPU baselines, as listed in Table 1. For the CPU baseline, we use Intel i7-6850K with single-precision *mkl_sparse_sp2m* SpGEMM function in Intel MKL [26]. To measure power consumption, we use the *Intel Performance Counter Monitor* tool [27]. For the GPU baseline, we use NVIDIA A100 with single-precision *cusparseScsrgemm2* SpGEMM function in cuSPARSE library [51]. Power consumption is measured with *nvidia-smi* [54].

4.2 Experimental Results

Figure 13 (A) and (B) shows the speedup and energy efficiency (i.e., FLOPS/Watt [15]) of Spaghetti, HARP, and GPU (cuSPARSE) normalized to CPU (MKL). The execution time and energy consumption in Spaghetti and HARP include the software-based tiling and hardware-based pseudo-tiling overheads, respectively. The energy consumptions of Spaghetti and HARP include static and dynamic energy used by SRAM components [33], multipliers and adders, and DRAM [3]. Additionally, in HARP, we include the energy consumption of the HARP controller. As the number of initial pseudo-tiles increases, fewer sub-tilings are required in certain datasets (e.g., *cit-Patents* and *cop20K_A*), which further improves the speedup and energy efficiency. As a result, on average, HARP-32, HARP-64, HARP-128, and HARP-256 achieve 31×, 32×, 33×, and 35× speedup and 551×, 555×, 595×, and 664× energy efficiency compared to MKL, respectively.

4.3 Analysis of Improvements

There are three factors contributing to the speedup and energy efficiency improvements in HARP over Spaghetti: (1) high accuracy of pseudo-tiling (2) low overheads of pseudo-tiling, and (3) ineffectual accesses elimination.

[3]We measure the CPU energy consumption of the software-based tiling in Spaghetti by using *Intel Performance Counter Monitor* tool. We obtain that the software-based tiling consumes approximately 2× the energy of MKL for conducting a SpGEMM operation. Thus, we conclude that the software-based tiling is impractical in terms of energy efficiency. In Figure 13 (B), because the CPU energy consumption of the software-based tiling is based on the real device, we only include the DRAM energy consumption of the software-based tiling in Spaghetti.

(1) High Accuracy of Pseudo-Tiling: The number of tiles in matrix A determines the number of accesses to matrix B. Therefore, the accuracy of tiling affects the performance and energy efficiency of the accelerator. Table 2 lists the number of tiles in the software-based tiling of Spaghetti and the number of final pseudo-tiles after super-tiling and sub-tiling in HARP. As the number of initial pseudo-tiles increases, HARP can analyze an input matrix in smaller granularity, thereby improving the accuracy of pseudo-tiling. As a result, the average relative error rates of HARP-32, HARP-64, HARP-128, and HARP-256 are 22.6%, 16.8%, 9.4%, and 5.8%, respectively. As listed for *email-Enron* in Table 2, since the number of initial pseudo-tiles in HARP is a power-of-2 and the input matrix size is non-power-of-2, there is a slight fluctuation in the number of final pseudo-tiles.

Table 2: The number of tiles in Spaghetti and HARP

Dataset	N	Spag.	HARP			
			32	**64**	**128**	**256**
2cubes_sphere	101K	14	16	16	15	14
amazon0312	401K	14	19	16	15	15
ca-CondMat	23K	3	3	3	3	3
cage12	130K	17	24	20	19	18
cit-Patents	4M	42	51	54	60	57
cop20k_A	121K	40	58	64	50	42
email-Enron	37K	36	**46**	**43**	**44**	**45**
filter3D	106K	45	64	64	52	47
m133-b3	200K	2	2	2	2	2
mario002	390K	7	7	7	7	7
offshore	260K	35	64	64	43	37
p2p-Gnutella31	63K	1	1	1	1	1
patents_main	241K	2	2	2	2	2
poisson3Da	14K	6	7	6	6	6
roadNet-CA	2M	9	11	10	9	9
scircuit	171K	5	5	5	5	5
web-Google	916K	31	35	34	33	33
webbase-1M	1M	38	51	45	45	44
wiki-Vote	8K	3	4	3	3	3

(2) Low Overheads of Pseudo-Tiling: By utilizing a lightweight hardware-based HARP controller, HARP achieves significantly low overheads for tiling compared to the software-based tiling in Spaghetti. Table 3 shows the area and power overheads of the HARP controller. Since the area and power overheads of outer product accelerators [57, 75] are approximately $20mm^2 \sim 90mm^2$ and 10 W, respectively, the overheads of a HARP controller are negligible.

Table 3: Area and power consumption of HARP controller, depending on the number of initial pseudo-tiles

	HARP			
	32	**64**	**128**	**256**
Tech.	45nm	45nm	45nm	45nm
Area	$0.24mm^2$	$0.26mm^2$	$0.29mm^2$	$0.38mm^2$
Power	39.1mW	40.7mW	42.6mW	45.5mW

In addition to a lightweight HARP controller, HARP has lower overheads for tiling than the software-based tiling. As discussed in Section 2.3, the software-based tiling incurs memory-intensive random accesses and copy operations. Since cache prefetchers in CPUs cannot work well for random accesses, these accesses are serialized, thereby increasing the execution time of the software-based tiling. (i.e., coupled data orchestration [59]). In contrast, because HARP sequentially accesses *ptrA*, *idxA*, *ptrB* arrays, and *RODs* during *Pseudo-Tiling Stage* and *Sub-Tiling Stage*, HARP can be implemented with FIFO (i.e., decoupled data orchestration [59]). As a result, memory accesses during these stages and *Pseudo-Tiling Context* updates can be performed in parallel, thereby decreasing the execution time of pseudo-tiling in HARP.

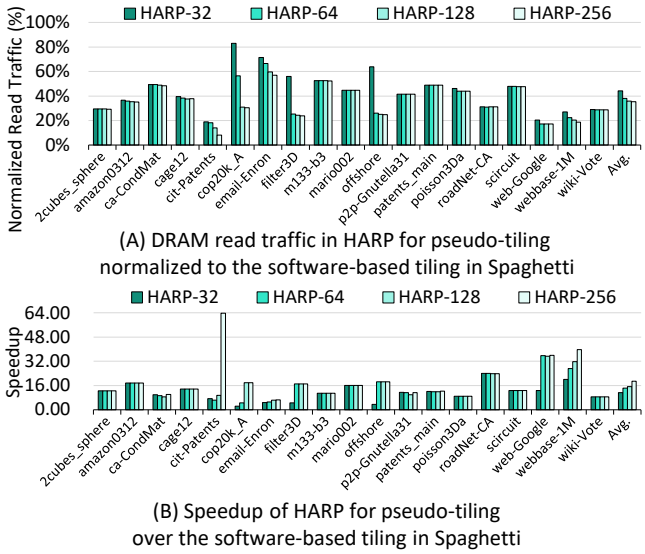

Figure 14: Comparison of DRAM read traffic and speedup between HARP and the software-based tiling.

Table 4: The number of *Sub-Tiling Stage* executions

Dataset	HARP			
	32	**64**	**128**	**256**
cit-Patents	13	17	15	0
cop20k_A	26	11	0	0
email-Enron	**7**	**7**	**6**	**7**
filter3D	32	0	0	0
offshore	32	0	0	0
web-Google	3	0	0	0
webbase-1M	10	7	8	6

*The unlisted datasets have no sub-tiling

Figure 14 (A) shows the DRAM read traffic in HARP for pseudo-tiling normalized to the software-based tiling in Spaghetti. As shown in Table 4, as the number of initial pseudo-tiles increases, fewer sub-tilings are required in certain datasets (e.g., *cit-Patents* and *cop20K_A*), which further decrease the DRAM read traffic in HARP. The average DRAM traffic reductions in HARP-32, HARP-64,

HARP-128, and HARP-256 are 56%, 62%, 64% and 65%, respectively. Figure 14 (B) shows the speedup of HARP for pseudo-tiling (i.e., the sum of execution times for *Pseudo-Tiling Stage*, *Super-Tiling Stage*, and *Sub-Tiling Stage*) over the software-based tiling in Spaghetti. Due to the low DRAM traffic and decoupled data orchestration in HARP, the average normalized speedups for pseudo-tiling in HARP-32, HARP-64, HARP-128, and HARP-256 are 11.3×, 14.3×, 15.4×, and 18.8×, respectively. Note that the speedup in tiling also affects energy efficiency because refresh energy (REF) and background energy (BG) consumptions in DRAM are proportional to the execution time [23, 34, 39–42].

(3) Ineffectual Accesses Elimination: By leveraging RODs, HARP can eliminate ineffectual accesses. However, even with the elimination of ineffectual accesses, RODs in HARP can increase DRAM read traffic during accelerator execution under a certain condition. For instance, assuming the dimensions of the matrices A and B are $N \times N$, there are T tiles in matrix A, and the numbers of non-zero elements in matrices A and B are nnz_A and nnz_B, respectively. In this scenario, the read traffic size for *ptrA/ptrB* and for *idxA/valA* and *idxB/valB* are $(T+1)\times(N+1)$ and $2nnz_A+2\times T\times nnz_B$, respectively, in a SpGEMM operation with tiled matrices. In contrast, if the number of effectual column-row pairs in an i^{th} tile in matrix A is K_i, the read traffic size for RODs is $\Sigma_{i=0}^{T} 4K_i$ while the read traffic size for *idxA/valA* and *idxB/valB* is changed by the ratio of effectual accesses to non-zero elements. In short, as the ratio of effectual accesses and effectual column-row pairs approaches 1, the traffic size for *idxA/valA* and *idxB/valB* in HARP becomes similar in Spaghetti, while the total size of RODs becomes larger than the size of *ptrA/ptrB* in Spaghetti.

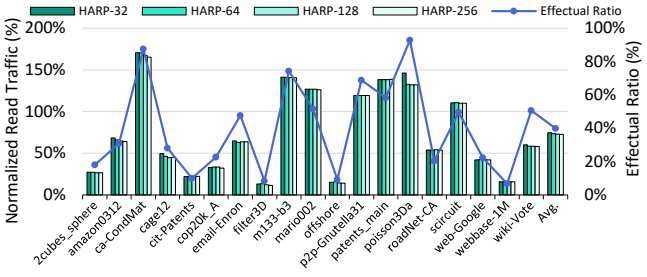

Figure 15: DRAM read traffic in HARP normalized to Spaghetti and the effectual ratios of non-zero elements.

Figure 15 shows the DRAM read traffic in HARP during a SpGEMM operation normalized to Spaghetti (the bar graphs) and the effectual ratio of non-zero elements (the line graph). In the majority of cases, as the effectual ratio decreases, the DRAM read traffic in HARP decreases. In the case of *email-Enron*, even though the effectual ratio of non-zero elements is 48%, the ratio of effectual column-row pair is 11%, thereby decreasing the number of RODs. As a result, the DRAM traffic reduction in HARP-32 for *email-Enron* is 35%. In summary, the average DRAM traffic reductions in HARP-32, HARP-64, HARP-128, and HARP-256 are 25.4%, 26.5%, 26.9%, and 27.3%, respectively. Note that even if DRAM read traffic in HARP during a SpGEMM operation can be increased, the execution time increase is not significant because accesses for RODs and the acceleration executions are decoupled and thus can be performed in parallel.

4.4 Analysis of Sub-Tiling Stage

In certain regions where non-zero elements are concentrated (e.g., power-law matrices [10]), the *Sub-Tiling Stage* can be recursively called. For instance, *webbase-1M* is a directed graph representing web connectivity and exhibits a power-law distribution in the number of non-zero elements per row [16, 52]. The average number of non-zero elements per row is 3.1 while 199 rows (0.02%) have more than 300 non-zero elements. Among these 199 rows, 92 rows are concentrated in the range of row index 7,000 to 19,000.

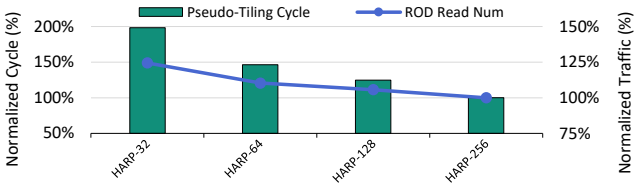

Figure 16: DRAM read traffic in HARP normalized to Spaghetti and the effectual ratios of non-zero elements.

Table 5: The number of *Sub-Tiling Stage* executions depending on the depth in *webbase-1M*

Depth	HARP			
	32	**64**	**128**	**256**
Depth-1	8	6	7	6
Depth-2	2	1	1	0

*Other datasets do not have a depth-2 *Sub-Tiling Stage*

Figure 16 presents the number of pseudo-tiling cycles and ROD reads in *Webbase-1M*, normalized to HARP-256 results, while Table 5 displays the number of *Sub-Tiling Stage* executions in *Webbase-1M* based on the sub-tiling depth. The depth-1 indicates the *Sub-Tiling Stage* called during regular pseudo-tile processing, while the depth-2 indicates the *Sub-Tiling Stage* called during sub-pseudo-tile processing. As the number of initial pseudo-tiles increases, the number of RODs in a pseudo-tile decreases, reducing the overheads of the *Sub-Tiling Stage*. Consequently, HARP-32, HARP-64, and HARP-128 exhibit 1.98x, 1.46x, and 1.25x cycle increase and 24%, 10%, and 6% more ROD read traffic compared to HARP-256, respectively. Additionally, since the depth-2 *Sub-Tiling Stage* utilizes the RODs in a sub-pseudo-tile, its overheads are smaller than the depth-1 *Sub-Tiling Stage*. For *webbase-1M*, the number of ROD reads for depth-2 in HARP-32, HARP-64, and HARP-128 accounts for 0.8%, 0.5%, and 0.4% of the total ROD read traffic.

4.5 Comparison of Inner Product Accelerators

Because HARP is based on the outer product accelerator, we additionally compare HARP against the state-of-the-art inner product accelerator (SIGMA) [60] for justification of HARP. Because SpGEMM is a memory-bound workload, the DRAM configuration highly affects the execution time. Thus, we choose SIGMA which has the open-source sst-stonne [50] cycle-accurate simulator to conduct experiments in the same DRAM simulator. Unlike outer

product accelerators, SIGMA uses a bitmap format to compress input matrices due to its architecture. The bitmap format consists of compression status bits for each element and a non-zero elements array. For SIGMA configuration, we use 4MB and 32MB oh-chip scratchpad memory for compression status bits and a non-zero elements array, respectively, and 16384 PEs with 1GHz frequency. We estimate the DRAM access times between DRAM and on-chip scratchpad memory by using Ramulator with the same DRAM configuration in Table 1. Because the metadata size in bitmap format is proportional to the matrix size regardless of the sparsity in the matrix, the maximum matrix size of SIGMA with 4MB on-chip scratchpad memory for compression status bits is 4096×4096. Thus, we synthesize R-MAT [10] matrices (i.e., the power-law matrix) and random matrices from 1e-03 to 1e-05 matrix density with 4096×4096 matrix size.

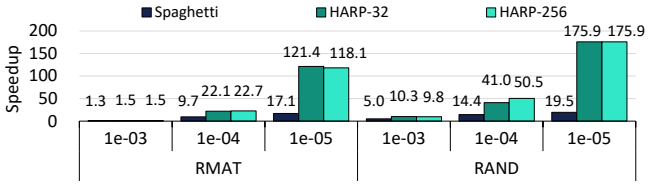

Figure 17: Speedup of Spaghetti and HARP over SIGMA.

Figure 17 shows the speedup of Spaghetti, HARP-32, and HARP-256, over SIGMA. The execution time of Spaghetti and HARP includes the software-based tiling and hardware-based pseudo-tiling overheads, respectively. The reasons for the speedup improvement are as follows: First, as the sparsity increases, the metadata size in CSC and CSR decreases because the number of *idx* elements is reduced, whereas the metadata size in bitmap format is constant. Thus, the more sparsity increases, the higher DRAM memory traffic is required in SIGMA than in outer product accelerators. Second, the inner product accelerator accesses elements in matrix B depending on the data distribution of matrix A. To this end, SIGMA selects some rows in matrix A and then accesses all compression status bits in matrix B to check the dependency. Consequently, the execution time of SIGMA is limited by dependency checking, even though the input matrices are entirely stored in the scratchpad memory. In contrast, outer product accelerators have independent multiplications for each column-row pair. Thus, as the sparsity increases, the execution time of outer product accelerators significantly decreases due to the reduction of multiplications. As a result, Spaghetti, HARP-32, and HARP-256 are 11.2×, 62.0×, and 63.1× faster than SIGMA on average.

5 DISCUSSION

5.1 The Impact of Skipping Hardware

As discussed in Section 2.3, Spaghetti with the software-based tiling has significant ineffectual accesses due to the absence of skipping hardware. To evaluate the impact of skipping hardware on Spaghetti with the software-based tiling, we incorporate skipping hardware into execution units in Spaghetti, adopting the same mechanism as used in HARP. Consequently, *ptrA* and *ptrB* elements are inspected by skipping hardware and then *idxA/valA* and *idxB/valB* elements are indirectly accessed only if both *ptrA* and *ptrB* elements indicate a non-zero column and row.

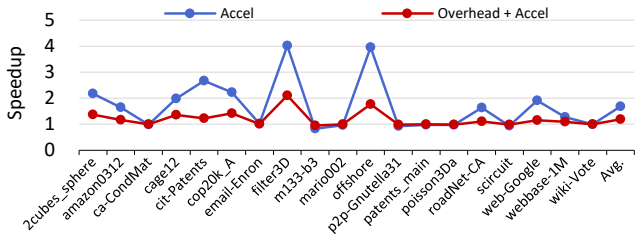

Figure 18: Comparisons of Spaghetti accelerator performance with and without skipping hardware.

The blue line and red line graphs in Figure 18 show the speedups of Spaghetti with skipping hardware over Spaghetti without skipping hardware. In most cases, the execution time for the accelerator alone (*Accel*) is decreased due to the elimination of the ineffectual accesses when employing skipping hardware. However, if there are insufficient ineffectual accesses (e.g., *m133-b3* and *p2p-Gnutella31*), the execution time is increased due to the indirect accesses. As discussed in Section 2.3, the software-based tiling overhead occupies 56.9% of the total execution time on average. Consequently, the speedup for Spaghetti with skipping hardware, including the software-based tiling overheads (*Overhead + Accel*), is 1.2× on average. In terms of DRAM read traffic reduction for *Overhead + Accel*, Spaghetti with skipping hardware reduces 18% traffic on average compared to Spaghetti without skipping hardware. Compared to HARP, the average speedups for HARP-32, HARP-64, HARP-128, and HARP-256 over Spaghetti with skipping hardware are 3.1×, 3.2×, 3.3×, and 3.5×, respectively.

5.2 Data Type

Unlike GEMM, the primary bottleneck of SpGEMM lies in the metadata of a compression format. For example, outer product accelerators can not handle a *val* element without its corresponding *idx* element because the *idx* element determines the index of the partial product. Consequently, the memory traffic reduction from using reduced precision (e.g., FP8, INT8, and INT4 [24, 28, 37, 56, 67]) does not significantly impact the execution time of outer product accelerators. In the case of HARP, *valA/valB* elements are not utilized during pseudo-tiling, making pseudo-tiling independent of data types. Our experiments demonstrate that the reduction in execution time for FP16, compared to FP32, is only 0.1% on average.

5.3 Memory Usage

Because HARP leverages RODs instead of *ptr* arrays, we discuss the peak memory usage of RODs and the memory usage measurement capability in HARP.

Peak Memory Usage of RODs: Because each tile in Spaghetti includes a *ptrA* array, the peak memory usage for *ptr* arrays in Spaghetti is proportional to the number of tiles. On the other hand, because a ROD in HARP indicates an effectual column-row pair in a pseudo-tile, the peak memory usage depends on the distribution of effectual column-row pairs in pseudo-tiles and the number of

final pseudo-tiles after super-tiling and sub-tiling. We calculate the peak memory usage in HARP by accounting for the entire reserved DRAM area used in RODs. For instance, when a column in matrix A has multiple non-zero elements throughout pseudo-tiles, we count the number of duplicated RODs.

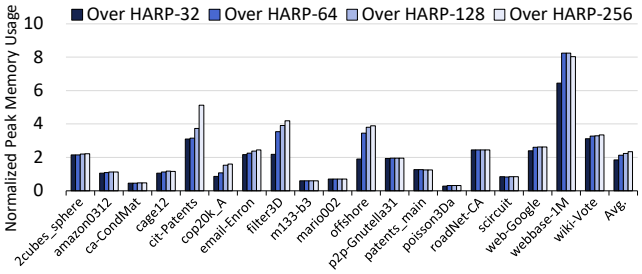

Figure 19: Normalized peak memory usage of HARP.

Figure 19 shows the peak memory usage of Spaghetti, normalized to HARP-32, HARP-64, HARP-128, and HARP-256, respectively. Note that because the peak memory usage is normalized to HARPs, the higher value means less memory usage in HARP. On average, Spaghetti consumes 1.8×, 2.1×, 2.2×, and 2.3× more memory than HARP-32, HARP-64, HARP-128, and HARP-256, respectively.

Memory Usage Measurement: HARP can measure the memory usage of RODs, similar to the *mkl_sparse_sp2m* function in MKL or the *cusparseScsrgemm2_bufferSizeExt* function in cuSPARSE library. To measure the memory usage of RODs, HARP simply performs operations similar to *Pseudo-Tiling Stage* and *Sub-Tiling Stage* to count the maximum number of RODs in each pseudo-tile. The average execution time for *Pseudo-Tiling Stage* and *Sub-Tiling Stage* in HARP-256 is 13.6% of the total execution time. Thus, the memory usage measurement overhead is reasonable.

6 RELATED WORK

Software-Based Matrix Tiling Methods: There are numerous software-based tiling methods to optimize SpGEMM operations at the algorithm level for supercomputing systems or distributed systems [1, 2, 5, 11, 31, 32, 44, 72]. For instance, Chen *et al.* [11] use software to partition a compressed input matrix to several cores in the Sunway TaihuLight supercomputer. Ballard *et al.* [5] propose the hypergraph model to optimize inter-processor communication cost in SpGEMM algorithms. Thus, none of these methods are designed for outer product accelerators and they are software-based tiling. In contrast, HARP is a hardware-based pseudo-tiling for outer product accelerators.

Outer Product Accelerators: For outer product accelerators, SCNN [58], OuterSPACE [57], MOSCON [53], SpArch [75], Spaghetti [22], and dual-side sparse tensor core [70] have been proposed. SCNN applies the outer product algorithm to convolution operations. OuterSPACE does not use an on-chip partial products buffer and therefore requires additional DRAM traffic for all partial products. MOSCON optimizes the load balancing and merging mechanism in OuterSPACE. SpArch uses an on-chip partial products buffer and condenses columns in matrix A to optimize the buffer usage. Thus, SpArch accesses several rows in matrix B for a condensed column, thereby degrading the input reuse of matrix B. Spaghetti tiles an input matrix by software to store all partial products in the on-chip buffer without spoiling input reuse. Dual-side sparse tensor core modifies the inner product operation of NVIDIA's tensor core [55] to the outer product operation, utilizing a bitmap format.

HARP can be applied to outer product based SpGEMM accelerators using CSC and CSR formats for matrices A and B. For instance, as OuterSPACE requires a large amount of memory usage to store entire partial products, HARP can control the amount of memory usage for partial products by supporting pseudo-tiling. In the case of SpArch, because SpArch re-fetches partial products, HARP can control the number of re-fetched partial products through pseudo-tiling.

Inner Product Accelerators: Several inner product accelerators [60, 63, 64, 71] and row-wise inner product accelerators [4, 25, 65, 73] have been proposed for SpGEMM operations. Inner product accelerators access one row in matrix A and several columns in matrix B based on the non-zero elements in matrix A to generate one output row. On the other hand, row-wise inner product accelerators access several rows in matrix B, instead of columns, as inner product accelerators. Consequently, the performance and energy efficiency of these accelerators depends on the access pattern in matrix B.

7 CONCLUSION

This paper exposes the limitations of the software-based tiling in outer product accelerators. To overcome these limitations, we propose hardware-based pseudo-tiling (HARP). To achieve low overheads in pseudo-tiling and eliminate ineffectual accesses, HARP utilizes a ROD to access an effectual column-row pair in a pseudo-tile. To improve the accuracy of the pseudo-tiles, HARP performs super-tiling and sub-tiling. As a result, HARP provides 4× speedup and 5× energy efficiency on average than the state-of-the-art outer product accelerator. Furthermore, we believe that our proposed ROD is not limited to outer product accelerators and has the potential to be applied in other domains that require accessing particular elements in sparse matrices stored in a compressed format. This may open up avenues for future research and development of efficient and accurate data access techniques.

ACKNOWLEDGMENTS

We thank the anonymous reviewers for their valuable feedback. This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. 2022R1A2C200632112).

REFERENCES

[1] Kadir Akbudak and Cevdet Aykanat. 2014. Simultaneous input and output matrix partitioning for outer-product–parallel sparse matrix-matrix multiplication. *SIAM Journal on Scientific Computing* 36, 5 (2014), C568–C590.

[2] Kadir Akbudak, Oguz Selvitopi, and Cevdet Aykanat. 2018. Partitioning models for scaling parallel sparse matrix-matrix multiplication. *ACM Transactions on Parallel Computing (TOPC)* 4, 3 (2018), 1–34.

[3] Jonathan Bachrach, Huy Vo, Brian Richards, Yunsup Lee, Andrew Waterman, Rimas Avižienis, John Wawrzynek, and Krste Asanović. 2012. Chisel: constructing hardware in a scala embedded language. In *DAC Design automation conference 2012*. IEEE, 1212–1221.

[4] Daehyeon Baek, Soojin Hwang, Taekyung Heo, Daehoon Kim, and Jaehyuk Huh. 2021. InnerSP: A Memory Efficient Sparse Matrix Multiplication Accelerator with Locality-Aware Inner Product Processing. In *2021 30th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE, 116–128.

[5] Grey Ballard, Alex Druinsky, Nicholas Knight, and Oded Schwartz. 2015. Hypergraph Partitioning for Parallel Sparse Matrix-Matrix Multiplication. In *Proceedings of the 27th ACM symposium on Parallelism in Algorithms and Architectures*. 86–88.

[6] Jeff Bolz, Ian Farmer, Eitan Grinspun, and Peter Schröder. 2003. Sparse matrix solvers on the GPU: conjugate gradients and multigrid. *ACM transactions on graphics (TOG)* 22, 3 (2003), 917–924.

[7] Aydin Buluç, Jeremy T Fineman, Matteo Frigo, John R Gilbert, and Charles E Leiserson. 2009. Parallel sparse matrix-vector and matrix-transpose-vector multiplication using compressed sparse blocks. In *Proceedings of the twenty-first annual symposium on Parallelism in algorithms and architectures*. 233–244.

[8] Aydın Buluç and John R Gilbert. 2011. The Combinatorial BLAS: Design, implementation, and applications. *The International Journal of High Performance Computing Applications* 25, 4 (2011), 496–509.

[9] Daniele Buono, Fabrizio Petrini, Fabio Checconi, Xing Liu, Xinyu Que, Chris Long, and Tai-Ching Tuan. 2016. Optimizing sparse matrix-vector multiplication for large-scale data analytics. In *Proceedings of the 2016 International Conference on Supercomputing*. 1–12.

[10] Deepayan Chakrabarti, Yiping Zhan, and Christos Faloutsos. 2004. R-MAT: A recursive model for graph mining. In *Proceedings of the 2004 SIAM International Conference on Data Mining*. SIAM, 442–446.

[11] Yuedan Chen, Guoqing Xiao, and Wangdong Yang. 2020. Optimizing partitioned CSR-based SpGEMM on the Sunway TaihuLight. *Neural Computing and Applications* 32, 10 (2020), 5571–5582.

[12] Timothy A Davis and Yifan Hu. 2011. The University of Florida sparse matrix collection. *ACM Transactions on Mathematical Software (TOMS)* 38, 1 (2011), 1–25.

[13] Robert D Falgout and Ulrike Meier Yang. 2002. hypre: A library of high performance preconditioners. In *International Conference on computational science*. Springer, 632–641.

[14] Siying Feng, Xin He, Kuan-Yu Chen, Liu Ke, Xuan Zhang, David Blaauw, Trevor Mudge, and Ronald Dreslinski. 2022. MeNDA: a near-memory multi-way merge solution for sparse transposition and dataflows. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 245–258.

[15] Wu-chun Feng and Kirk Cameron. 2007. The green500 list: Encouraging sustainable supercomputing. *Computer* 40, 12 (2007), 50–55.

[16] Salvatore Filippone, Valeria Cardellini, Davide Barbieri, and Alessandro Fanfarillo. 2017. Sparse matrix-vector multiplication on GPGPUs. *ACM Transactions on Mathematical Software (TOMS)* 43, 4 (2017), 1–49.

[17] Jeremy Fowers, Kalin Ovtcharov, Karin Strauss, Eric S Chung, and Greg Stitt. 2014. A high memory bandwidth fpga accelerator for sparse matrix-vector multiplication. In *2014 IEEE 22nd Annual International Symposium on Field-Programmable Custom Computing Machines*. IEEE, 36–43.

[18] Sameh Galal and Mark Horowitz. 2010. Energy-efficient floating-point unit design. *IEEE Transactions on computers* 60, 7 (2010), 913–922.

[19] Christina Giannoula, Ivan Fernandez, Juan Gómez Luna, Nectarios Koziris, Georgios Goumas, and Onur Mutlu. 2022. SparseP: Towards efficient sparse matrix vector multiplication on real processing-in-memory architectures. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 6, 1 (2022), 1–49.

[20] John R Gilbert, Steve Reinhardt, and Viral B Shah. 2006. High-performance graph algorithms from parallel sparse matrices. In *International Workshop on Applied Parallel Computing*. Springer, 260–269.

[21] Song Han, Huizi Mao, and William J Dally. 2015. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149* (2015).

[22] Reza Hojabr, Ali Sedaghati, Amirali Sharifian, Ahmad Khonsari, and Arrvindh Shriraman. 2021. Spaghetti: Streaming accelerators for highly sparse gemm on fpgas. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 84–96.

[23] Jeongkyu Hong, Hyeonggyu Kim, and Soontae Kim. 2018. EAR: ECC-aided refresh reduction through 2-D zero compression. In *Proceedings of the 27th International Conference on Parallel Architectures and Compilation Techniques*. 1–11.

[24] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2017. Quantized neural networks: Training neural networks with low precision weights and activations. *The Journal of Machine Learning Research* 18, 1 (2017), 6869–6898.

[25] Ranggi Hwang, Minhoo Kang, Jiwon Lee, Dongyun Kam, Youngjoo Lee, and Minsoo Rhu. 2023. GROW: A Row-Stationary Sparse-Dense GEMM Accelerator for Memory-Efficient Graph Convolutional Neural Networks. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 42–55.

[26] IntelMKL. 2023. Intel oneapi math kernel library (oneMKL). https://www.intel.com/content/www/us/en/develop/documentation/oneapi-programming-guide/top/api-based-programming/intel-oneapi-math-kernel-library-onemkl.html

[27] IntelPCM. 2022. Intel® Performance Counter Monitor - A better way to measure CPU Utilization. https://www.intel.com/content/www/us/en/developer/articles/technical/performance-counter-monitor.html#license

[28] Myeongjae Jang, Jinkwon Kim, Jesung Kim, and Soontae Kim. 2022. Encore compression: Exploiting narrow-width values for quantized deep neural networks. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1503–1508.

[29] Konstantinos Kanellopoulos, Nandita Vijaykumar, Christina Giannoula, Roknoddin Azizi, Skanda Koppula, Nika Mansouri Ghiasi, Taha Shahroodi, Juan Gomez Luna, and Onur Mutlu. 2019. Smash: Co-designing software compression and hardware-accelerated indexing for efficient sparse matrix operations. In *Proceedings of the 52nd annual IEEE/ACM international symposium on microarchitecture*. 600–614.

[30] Mincheol Kang, Wonyoung Lee, Jinkwon Kim, and Soontae Kim. 2022. PR-SSD: Maximizing Partial Read Potential by Exploiting Compression and Channel-Level Parallelism. *IEEE Trans. Comput.* 72, 3 (2022), 772–785.

[31] Oguz Kaya, Ramakrishnan Kannan, and Grey Ballard. 2018. Partitioning and communication strategies for sparse non-negative matrix factorization. In *Proceedings of the 47th International Conference on Parallel Processing*. 1–10.

[32] Oguz Kaya and Bora Uçar. 2015. Scalable sparse tensor decompositions in distributed memory systems. In *SC'15: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–11.

[33] Jinkwon Kim, Seokin Hong, Jeongkyu Hong, and Soontae Kim. 2020. Cid: Co-architecting instruction cache and decompression system for embedded systems. *IEEE Trans. Comput.* 70, 7 (2020), 1132–1145.

[34] Jinkwon Kim, Mincheol Kang, Jeongkyu Hong, and Soontae Kim. 2022. Exploiting inter-block entropy to enhance the compressibility of blocks with diverse data. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 1100–1114.

[35] Yoongu Kim, Weikun Yang, and Onur Mutlu. 2015. Ramulator: A fast and extensible DRAM simulator. *IEEE Computer architecture letters* 15, 1 (2015), 45–49.

[36] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).

[37] Raghuraman Krishnamoorthi. 2018. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342* (2018).

[38] Pran Kurup and Taher Abbasi. 2012. *Logic synthesis using Synopsys®*. Springer Science & Business Media.

[39] Yebin Lee, Hyeonggyu Kim, Seokin Hong, and Soontae Kim. 2017. Partial row activation for low-power dram system. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 217–228.

[40] Yebin Lee and Soontae Kim. 2015. CLAP: clustered look-ahead prefetching for energy-efficient DRAM system. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 24, 5 (2015), 1770–1782.

[41] Yebin Lee and Soontae Kim. 2016. RAMS: DRAM rank-aware memory scheduling for energy saving. *IEEE Trans. Comput.* 65, 10 (2016), 3210–3216.

[42] Yebin Lee, Soontae Kim, Seokin Hong, and Jongmin Lee. 2013. Skinflint DRAM system: Minimizing DRAM chip writes for low power. In *2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 25–34.

[43] Jure Leskovec, Lada A Adamic, and Bernardo A Huberman. 2007. The dynamics of viral marketing. *ACM Transactions on the Web (TWEB)* 1, 1 (2007), 5–es.

[44] Kenli Li, Wangdong Yang, and Keqin Li. 2014. Performance analysis and optimization for SpMV on GPU using probabilistic modeling. *IEEE Transactions on Parallel and Distributed Systems* 26, 1 (2014), 196–205.

[45] Sheng Li, Ke Chen, Jung Ho Ahn, Jay B Brockman, and Norman P Jouppi. 2011. CACTI-P: Architecture-level modeling for SRAM-based structures with advanced leakage reduction techniques. In *2011 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 694–701.

[46] Shang Li, Zhiyuan Yang, Dhiraj Reddy, Ankur Srivastava, and Bruce Jacob. 2020. DRAMsim3: a cycle-accurate, thermal-capable DRAM simulator. *IEEE Computer Architecture Letters* 19, 2 (2020), 106–109.

[47] Baoyuan Liu, Min Wang, Hassan Foroosh, Marshall Tappen, and Marianna Pensky. 2015. Sparse convolutional neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 806–814.

[48] Weifeng Liu and Brian Vinter. 2015. CSR5: An efficient storage format for cross-platform sparse matrix-vector multiplication. In *Proceedings of the 29th ACM on International Conference on Supercomputing*. 339–350.

[49] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. 2005. Pin: building customized program analysis tools with dynamic instrumentation. *Acm sigplan notices* 40, 6 (2005), 190–200.

[50] Francisco Muñoz-Martínez, José L Abellán, Manuel E Acacio, and Tushar Krishna. 2021. STONNE: Enabling cycle-level microarchitectural simulation for dnn inference accelerators. In *2021 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 201–213.

[51] Maxim Naumov, L Chien, Philippe Vandermersch, and Ujval Kapasi. 2010. Cusparse library. In *GPU Technology Conference*.

[52] Yuyao Niu, Zhengyang Lu, Haonan Ji, Shuhui Song, Zhou Jin, and Weifeng Liu. 2022. TileSpGEMM: a tiled algorithm for parallel sparse general matrix-matrix multiplication on GPUs. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 90–106.

[53] G Noble, S Nalesh, and S Kala. 2023. MOSCON: Modified Outer Product based Sparse Matrix-Matrix Multiplication Accelerator with Configurable Tiles. In *2023 36th International Conference on VLSI Design and 2023 22nd International Conference on Embedded Systems (VLSID)*. IEEE, 264–269.

[54] Nvidia. 2022. Nvidia System Management Interface. https://developer.nvidia.com/nvidia-system-management-interface

[55] N NVIDIA. 2020. NVIDIA A100 tensor core GPU architecture. *Volume 1.0: Whitepaper, Part* 1 (2020), 82.

[56] NVIDIA NVIDIA. 2022. H100 tensor core GPU architecture overview.

[57] Subhankar Pal, Jonathan Beaumont, Dong-Hyeon Park, Aporva Amarnath, Siying Feng, Chaitali Chakrabarti, Hun-Seok Kim, David Blaauw, Trevor Mudge, and Ronald Dreslinski. 2018. Outerspace: An outer product based sparse matrix multiplication accelerator. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 724–736.

[58] Angshuman Parashar, Minsoo Rhu, Anurag Mukkara, Antonio Puglielli, Rangharajan Venkatesan, Brucek Khailany, Joel Emer, Stephen W Keckler, and William J Dally. 2017. SCNN: An accelerator for compressed-sparse convolutional neural networks. *ACM SIGARCH computer architecture news* 45, 2 (2017), 27–40.

[59] Michael Pellauer, Yakun Sophia Shao, Jason Clemons, Neal Crago, Kartik Hegde, Rangharajan Venkatesan, Stephen W Keckler, Christopher W Fletcher, and Joel Emer. 2019. Buffets: An efficient and composable storage idiom for explicit decoupled data orchestration. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 137–151.

[60] Eric Qin, Ananda Samajdar, Hyoukjun Kwon, Vineet Nadella, Sudarshan Srinivasan, Dipankar Das, Bharat Kaul, and Tushar Krishna. 2020. Sigma: A sparse and irregular gemm accelerator with flexible interconnects for dnn training. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 58–70.

[61] Oguz Selvitopi, Md Taufique Hussain, Ariful Azad, and Aydın Buluç. 2020. Optimizing high performance Markov clustering for pre-exascale architectures. In *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 116–126.

[62] Yakun Sophia Shao, Sam Likun Xi, Vijayalakshmi Srinivasan, Gu-Yeon Wei, and David Brooks. 2016. Co-designing accelerators and SoC interfaces using gem5-Aladdin. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–12.

[63] Linghao Song, Yuze Chi, Licheng Guo, and Jason Cong. 2022. Serpens: A high bandwidth memory based accelerator for general-purpose sparse matrix-vector multiplication. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 211–216.

[64] Linghao Song, Yuze Chi, Atefeh Sohrabizadeh, Young-kyu Choi, Jason Lau, and Jason Cong. 2022. Sextans: A Streaming Accelerator for General-Purpose Sparse-Matrix Dense-Matrix Multiplication. In *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 65–77.

[65] Nitish Srivastava, Hanchen Jin, Jie Liu, David Albonesi, and Zhiru Zhang. 2020. Matraptor: A sparse-sparse matrix multiplication accelerator based on row-wise product. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 766–780.

[66] Manuel Then, Moritz Kaufmann, Fernando Chirigati, Tuan-Anh Hoang-Vu, Kien Pham, Alfons Kemper, Thomas Neumann, and Huy T Vo. 2014. The more the merrier: Efficient multi-source graph traversal. *Proceedings of the VLDB Endowment* 8, 4 (2014), 449–460.

[67] Vincent Vanhoucke, Andrew Senior, and Mark Z. Mao. 2011. Improving the speed of neural networks on CPUs. In *Deep Learning and Unsupervised Feature Learning Workshop, NIPS 2011*.

[68] P. Virtanen, R. Gommers, T.E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S.J. van Der Walt, M. Brett, J. Wilson, K.J. Millman, N. Mayorov, A.R.J. Nelson, E. Jones, R. Kern, E. Larson, C.J. Carey, I. Polat, Y. Feng, E.W. Moore, J. Vanderplas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E.A. Quintero, C.R. Harris, A.M. Archibald, A.H. Ribeiro, F. Pedregosa, P. Van Mulbregt, A. Vijaykumar, A.P. Bardelli, A. Rothberg, A. Hilboll, A. Kloeckner, A. Scopatz, A. Lee, A. Rokem, C.N. Woods, C. Fulton, C. Masson, C. Häggström, C. Fitzgerald, D.A. Nicholson, D.R. Hagen, D.V. Pasechnik, E. Olivetti, E. Martin, E. Wieser, F. Lenders, Fabrice Silva, F. Wilhelm, G. Young, G.A. Price, G.-L. Ingold, G.E. Allen, G.R. Lee, H. Audren, Irvin Probst, J.P. Dietrich, J. Silterra, J.T. Webber, J. Slavič, J. Nothman, J. Buchner, J. Kulick, J.L. Schönberger, J.V. De Miranda Cardoso, J. Reimer, J. Harrington, J.L.C. Rodríguez, J. Nunez-Iglesias, J. Kuczynski, K. Tritz, M. Thoma, M. Newville, M. Kümmerer, M. Bolingbroke, M. Tartre, M. Pak, N.J. Smith, N. Nowaczyk, N. Shebanov, O. Pavlyk, P.A. Brodtkorb, P. Lee, R.T. Mcgibbon, R. Feldbauer, S. Lewis, S. Tygier, S. Sievert, S. Vigna, S. Peterson, S. More, T. Pudlik, T. Oshima, T.J. Pingel, T.P. Robitaille, T. Spura, T.R. Jones, T. Cera, T. Leslie, T. Zito, T. Krauss, U. Upadhyay, Y.O. Halchenko, and Y. Vázquez-Baeza. 2020. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature methods* 17, 3 (2020), 261–272.

[69] Hanrui Wang, Jiacheng Yang, Hae-Seung Lee, and Song Han. 2018. Learning to design circuits. *arXiv preprint arXiv:1812.02734* (2018).

[70] Yang Wang, Chen Zhang, Zhiqiang Xie, Cong Guo, Yunxin Liu, and Jingwen Leng. 2021. Dual-side sparse tensor core. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1083–1095.

[71] Shiyao Xu, Jingfei Jiang, Jinwei Xu, Chaorun Liu, Yuanhong He, Xiaohang Liu, and Lei Gao. 2022. Sparkle: A High Efficient Sparse Matrix Multiplication Accelerator for Deep Learning. In *2022 IEEE 40th International Conference on Computer Design (ICCD)*. IEEE, 479–486.

[72] Wangdong Yang, Kenli Li, Zeyao Mo, and Keqin Li. 2014. Performance optimization using partitioned SpMV on GPUs and multicore CPUs. *IEEE Trans. Comput.* 64, 9 (2014), 2623–2636.

[73] Guowei Zhang, Nithya Attaluri, Joel S Emer, and Daniel Sanchez. 2021. Gamma: Leveraging Gustavson's algorithm to accelerate sparse matrix multiplication. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 687–701.

[74] Yudong Zhang, Lenan Wu, Geng Wei, and Shuihua Wang. 2011. A novel algorithm for all pairs shortest path problem based on matrix multiplication and pulse coupled neural network. *Digital Signal Processing* 21, 4 (2011), 517–521.

[75] Zhekai Zhang, Hanrui Wang, Song Han, and William J Dally. 2020. Sparch: Efficient architecture for sparse matrix multiplication. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 261–274.